Laboratorio di programmazione per sistemi ciberfisici

1. Introduzione

Enrico Martini Michele Boldo

A.A. 2026/2027

Organizzazione del corso

Matricole pari

Teoria
Nicola Bombieri

Laboratorio
Michele Boldo

Contatto
`michele.boldo@univr.it`

Matricole dispari

Teoria
Bogdan Maris

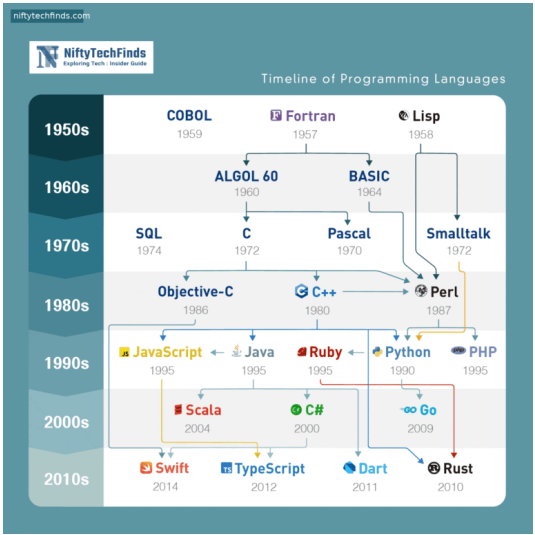
Laboratorio
Enrico Martini

Contatto
`enrico.martini@univr.it`

Ricevimento – Laboratorio

- Al termine di ogni lezione di laboratorio
- Su appuntamento via e-mail

Perchè imparare il linguaggio C



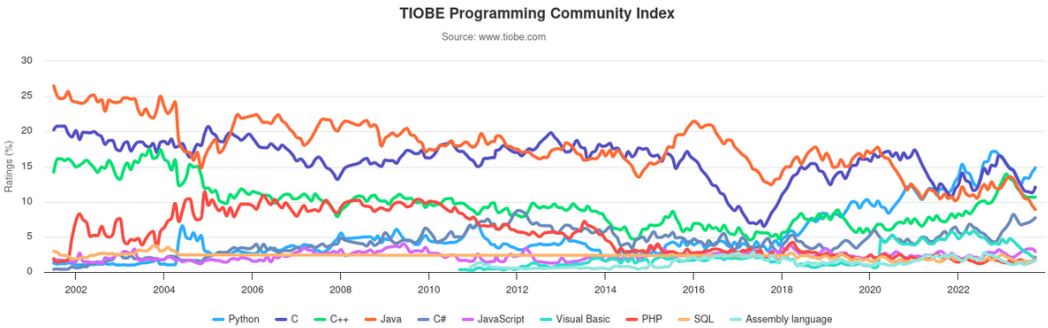
Perchè imparare il linguaggio C

- Permette di lavorare vicino all'Hardware (HW)
- Offre controllo sull'uso della memoria e delle risorse
- Utilizzato nella programmazione di **sistemi ciberfisici** e dispositivi embedded

Definizione

Un **sistema ciberfisico** combina elaborazione software e interazione con il mondo fisico.

Perchè imparare il linguaggio C



Sistemi di elaborazione

- **Software applicativo (SW)**: i programmi che eseguiremo
- **Sistema Operativo (OS)**: Ubuntu, Windows, etc
- **Firmware (FW)**: programma che gestisce le componenti fisiche
- **Compenenti fisici della macchina (HW)**: CPU, GPU, RAM, etc

Linux

- Linux è un **kernel** [1] open source basato su Unix, ovvero il cuore di alcuni sistemi operativi che gestisce risorse hardware e software.
- È usato in server, computer desktop, smartphone (Android), dispositivi IoT e supercomputer.
- Principali caratteristiche: gratuito, sicuro, stabile e altamente personalizzabile.



[1] Nell'uso comune il nome Linux viene spesso esteso ai sistemi operativi costruiti attorno a questo kernel

Ubuntu

- Ubuntu è una distribuzione (distro) di Linux, cioè una **versione completa del sistema operativo** basata sul kernel Linux con strumenti e applicazioni.
- Include interfaccia grafica, programmi preinstallati e aggiornamenti frequenti.

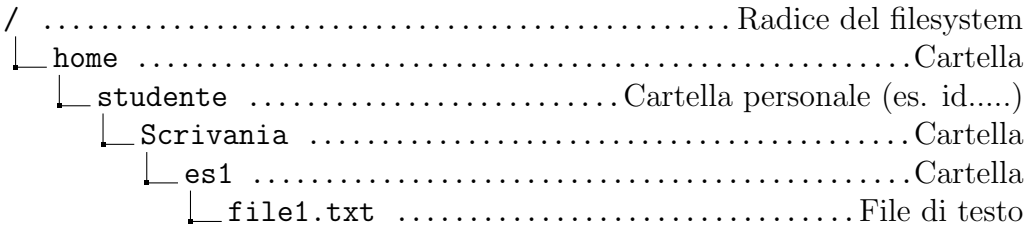


File System

- **File**: contenintore logico di dati o istruzioni
- **Directory**: contenitore logico di più file
- **Collegamenti**: riferimento ad oggetti nel file system

Struttura del File System

Struttura delle cartelle



Riferimenti e percorsi

Riferimenti veloci

/ radice ~ home . cartella corrente .. superiore

Assoluto — parte dalla radice.

```
/home/studente/Scrivania/es1/file1.txt
```

Relativo — parte dalla cartella corrente.
Se siamo in `/home/studente/Scrivania`:

```
es1/file1.txt
```

I due percorsi indicano **lo stesso file**.

La Shell Bash (Terminale)

Interfaccia del Sistema Operativo con l'utente

- Parte del sistema operativo
- E' un iterprete di comandi



Comandi Di Navigazione

```
pwd              # mostra la cartella in cui si trova l'utente
mkdir <nome>     # crea una nuova cartella
rmdir <nome>     # elimina la cartella (solo se vuota)

cd <percorso>    # sposta l'utente al percorso specificato
cd ..            # sposta l'utente alla cartella superiore
cd /             # sposta l'utente alla cartella base (root)
cd ~             # sposta l'utente alla cartella base dell'utente

ls <percorso>    # elenca i file contenuti nella cartella <percorso>
ls               # elenca i file contenuti nella cartella corrente
```

Seguiamo la posizione corrente

Posizione iniziale

`/home/studente`

Entriamo nella Scrivania

```
cd Scrivania
pwd
```

→ `/home/studente/Scrivania`

Torniamo alla cartella superiore

```
cd ..
pwd
```

→ `/home/studente`

Comandi Di Gestione File

```
cp <sorgente> <destinazione>    # copia file/directory
mv <sorgente> <destinazione>    # sposta file/directory (oppure rinominare)
rm <percorso>                   # cancella file
touch <filename>                # crea un file vuoto
cat <filename>                  # stampa il contenuto del file
locate <filename>               # trova il file nel sistema
```

Capire il risultato di un comando

Il silenzio è normale

Un comando riuscito può non stampare nulla. Verifica con `ls`, `pwd` o `cat`.

`No such file or directory` Controlla nome, percorso e cartella corrente.

`Permission denied` Controlla i permessi di accesso.

`command not found` Controlla il nome del comando e che sia installato.

Attenzione ai nomi

In Linux, normalmente: `Prova.txt` $\neq$ `prova.txt`.

Comandi generici

```
1 echo "Ciao"  # stampa un messaggio
2 man ls        # apre il manuale di ls
3 history       # mostra i comandi precedenti
4 clear         # pulisce lo schermo
5 du -h         # spazio occupato da file e cartelle
6 df -h         # spazio usato e disponibile nei filesystem
```

Scorciatoie utili

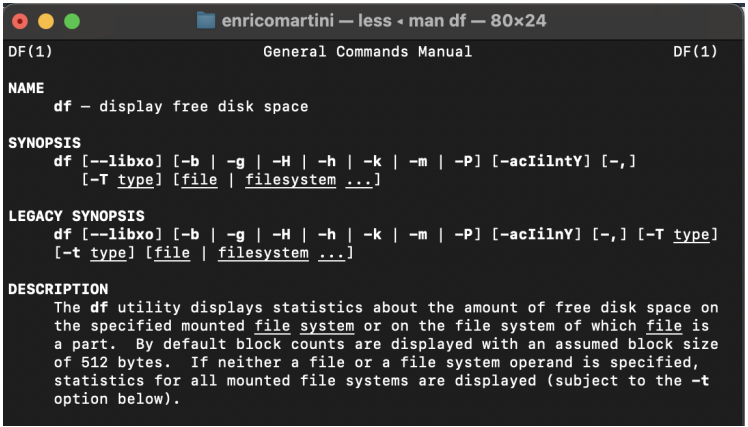
Tab (tabulazione): completa nomi e percorsi.

Freccia su (↑): richiama un comando precedente.

Ctrl+C (premuti insieme): normalmente interrompe il comando.

Esempio

```
1 man df
```



Editor di testo

Un editor di testo è un programma che permette di creare e modificare file di testo semplice (senza formattazione).

Da terminale:

- vim
- nano

Grafici (GUI):

- gedit
- vs code

Vim

Come lanciarlo da terminale:

```
vim <filename>
```

Ha 5 comandi base:

- `i` entra in modalità inserimento
- `ESC` esci dalla modalità d'inserimento
- `:w` salva il file
- `:q` chiudi il file
- `:wq` salva e chiudi il file

Nano

Come lanciarlo da terminale:

```
1 nano <filename>
```

Ha 2 comandi base:

- `Ctrl+o` salva il file
- `Ctrl+x` chiudi il file

Gedit

Gedit è un editor grafico per creare e modificare file di testo, sorgenti C e script.

Aprire un file dal terminale

```
gedit prova.txt
```

Se il file non esiste, viene creato al primo salvataggio.

Scorciatoie utili

Ctrl+O: apre un file.

Ctrl+S: salva le modifiche.

Ctrl+Z: annulla l'ultima modifica.

Script Bash

Uno **script** è un file di testo con comandi eseguiti in sequenza, per automatizzare operazioni ripetitive

Il file `script.sh`

```
1 #!/bin/bash
2 # Un primo script
3 echo "Ciao Mondo!"
4 echo "Hello World!"
```

`#` introduce un **commento**.

`#!/bin/bash` indica l'interprete da usare nell'avvio diretto.

Esecuzione con Bash

```
1 bash script.sh
```

Esercizio 1

1. Crea una cartella chiamata `es1` nella **Scrivania**
2. Entra nella cartella e crea un file vuoto `file1.txt`
3. Rinomina il file in `file2.txt`
4. Cancella il file `file2.txt`
5. Torna sulla Scrivania
6. Cancella la cartella `es1`

Esercizio 2

1. Crea una cartella chiamata `es2` nella **Scrivania**
2. Entra nella cartella e crea 3 file vuoti `a.txt`, `b.txt`, `c.txt`
3. Rinomina il file `c.txt` in `d.txt`
4. Inserisci il testo "*hello*" all'interno del file `a.txt` con vim
5. Inserisci il testo "*ciao*" all'interno del file `b.txt` con nano
6. Visualizza il testo contenuto all'interno del file `a.txt`
7. Stampa la data, il giorno e l'ora attuali utilizzando il comando `date`

Esercizio 3

Crea, utilizzando un editor a tua scelta, uno script bash nella cartella **Scrivania** chiamato `script.sh`, che stampa la seguenti frasi:

```
1 Ciao Mondo!
2 Hello World!
3 Oggi è Mar  7 Ott 2025 10:17:15 CEST
```

NB: per inserire l'ora precisa, utilizza il comando `$(date)`

Esercizio 4

Crea, utilizzando un editor di testo a tua scelta, uno script bash che risolva l'esercizio 2.